Style Radar

SQUAD GUATOQUE JIRAGUAS (GJ)
DOSW · SPRINT 02 · V2 · 2026

SQUAD GUATOQUE JIRAGUAS · ESPECIFICACIÓN TÉCNICA V2 · SPRINT 02

DOCUMENTO TÉCNICO V2

Contexto, arquitectura hexagonal, endpoints, eventos consumidos y publicados, modelo de datos y decisiones técnicas del microservicio **GJ-Notification-Service** (Puerto 8300 · Paquete 3) al cierre del Sprint 2 en StyleRadar.

MS-NOTIFICACIONES :8300 · PAQUETE 3 OFICIAL · ARQUITECTURA HEXAGONAL · POSTGRESQL 16 + FLYWAY V1-V3 · SPRING BOOT 3.2.5 · JAVA 21 · RABBITMQ 3.13.7 AZURE TLS :5671 · 8 COLAS + 3 REINTENTOS + DLQ · SSE · CAMPANA EN TIEMPO REAL · JACOCO >= 85 % · NEWMAN 8/8

FICHA TÉCNICA · ARQUITECTURA DE SOFTWARE · V2

PROYECTO / ASIGNATURA
StyleRadar · DOSW 2026

MICROSERVICIO ASIGNADO
GJ-Notification-Service (Puerto 8300 · Paquete 3)

LÍDER DE SQUAD
Paula Diaz

RESPONSABLES
Jose Martinez (Líder Backend) · Oscar Lasso (Desarrollo)

SR · DOSW · 2026

00 ESTRUCTURA DEL DOCUMENTO (V2)

Este documento reúne la especificación técnica v2 del microservicio **GJ-Notification-Service** (`ms-notificaciones`, puerto `8300`) al cierre del Sprint 2 y responde a la tarea **GJ-99** (continuación de GJ-28) del tablero del Squad Guatoque Jiraguas.

TABLA DE CONTENIDO OFICIAL

SECCIÓN	TÍTULO Y ALCANCE	DESCRIPCIÓN TÉCNICA DE LOS CONTENIDOS (ACTUALIZACIÓN V2)
01	**Contexto y drivers**	Problema, bounded context, atributos de calidad (ISO/IEC 25010), restricciones y estado de implementación por sprint.
02	**Arquitectura hexagonal**	Capas, diagrama de componentes actualizado (`:8300` y `:8220`), flujo de una petición, estructura de paquetes y deuda arquitectónica.
03	**Endpoints y Swagger**	Contrato REST en `:8300`, seguridad JWT, códigos de error (incluidos `403` de alerta ajena y `503` de catálogo) y evidencia de Swagger.
04	**Eventos**	8 eventos consumidos y 1 publicado, regla de reposición con `stockAnterior` (GJ-45), broker RabbitMQ en Azure con TLS `:5671` (GJ-91), 3 reintentos y DLQ (GJ-50).
05	**Modelo de datos**	Esquema PostgreSQL con Flyway (`V1`, `V2`, `V3__add_deduplication_key.sql`), restricciones e índices.
06	**Decisiones técnicas**	Metodología multicriterio y registros ADR-001 a ADR-006, incluido el broker RabbitMQ en Azure VM.
07	**Riesgos y mitigación**	Riesgos mitigados en el Sprint 2 y controles de calidad vigentes (CI, JaCoCo >= 85 %, Newman).
08	**Conclusiones y equipo**	Stack con versiones fijas, verificación realizada en Sprint 2, pendientes de Sprint 3 y responsables.

Cambios frente a la v1: migración al puerto oficial `8300` y catálogo en `8220` (**GJ-90**); aprovisionamiento del broker RabbitMQ `3.13.7` en VM de Azure (`styleradar-rabbitmq-qa.northcentralus.cloudapp.azure.com:5671` con TLS, **GJ-91**); regla de reposición real con `stockAnterior <= 0 && stockActual > 0` (**GJ-45**); octava cola `PrendaRetirada`; política de **3 reintentos exponenciales** antes de enviar a `styleradar.notificaciones.dlq` e idempotencia con `clave_deduplicacion` (**GJ-50**); y colección Postman verificada con Newman en CI (**GJ-52**, commit `f88c0cd`).

01 CONTEXTO Y DRIVERS DEL MICROSERVICIO

StyleRadar es una plataforma de moda local para Bogotá con inventario multi-sede. Este servicio avisa a cada usuario cuando ocurre algo relevante sin obligarlo a volver a buscar.

1.1 PLANTEAMIENTO DEL PROBLEMA

Un comprador que encuentra una prenda agotada, o que guardó una búsqueda, necesita enterarse cuando esa condición cambia. Al pasar de monolito a microservicios, consultar periódicamente las tablas de inventario violaría **Database-per-Service**. Por eso `ms-notificaciones` reacciona a eventos de otros dominios, evalúa los criterios en memoria y conserva copias locales de la información que necesita.

QUÉ HACE ESTE MICROSERVICIO	QUÉ NO HACE (DELEGADO)
Gestiona alertas `ITEM_AGOTADO` y `BUSQUEDA_GUARDADA` bajo `/api/v1/usuarios/me`.	No autentica ni emite tokens de sesión: lo hace el servicio de identidad (`ms-usuarios`).
Consulta disponibilidad síncrona en `ms-item-catalogo` (`:8220`) al crear una alerta por prenda.	No modifica inventario ni catálogo; si el catálogo cae responde `503 Service Unavailable`.
Genera notificaciones de reposición de stock, búsquedas, mensajes, donaciones y moderación.	No guarda conversaciones ni historial de chat: lo hace `ms-chat`.
Mantiene copias locales de búsquedas guardadas y estados de cuenta (`ACTIVA`, `INACTIVA`, `SUSPENDIDA`, `ELIMINADA`).	No accede a tablas de otros servicios ni declara llaves foráneas externas.
Entrega la bandeja, el contador y la campana en tiempo real por SSE.	No gestiona pagos ni pasarelas.

1.2 ATRIBUTOS DE CALIDAD CLAVE (ISO/IEC 25010)

ATRIBUTO	ESCENARIO OBJETIVO	IMPACTO EN EL DISEÑO (SPRINT 2)
Fiabilidad	Un evento repetido no duplica el aviso de un mismo ciclo de reposición.	Columna `clave_deduplicacion` con restricción `uk_notificacion_deduplicacion` (`V3`) y captura de `DataIntegrityViolationException`.
Tolerancia a fallos	Un fallo transitorio se reintenta y un payload inválido va directo a la DLQ.	`RabbitRetryConfig` (3 intentos: 1 s, 2 s, 4 s) + `AmqpRejectAndDontRequeueException` hacia `styleradar.dlx` / `styleradar.notificaciones.dlq`.
Eficiencia	La campana recibe novedades sin sondeo periódico.	Server-Sent Events (`SseEmitter`) por usuario autenticado.
Seguridad	Cada usuario opera solo sobre sus recursos y el broker cifra el tráfico.	JWT HS256 stateless (`JwtSecretValidator` exige secreto en prod) y AMQPS TLS en el puerto `5671`.
Mantenibilidad	Las reglas se prueban sin levantar Spring.	Dominio puro, puertos e interfaz + `Impl`; cobertura JaCoCo >= 85 % verificada en CI.
Portabilidad	El servicio se levanta y verifica con un comando.	Dockerfile multi-stage (puerto `8300`) y `docker compose up` con Newman en CI.

1.3 RESTRICCIONES TÉCNICAS DEL PROYECTO

- Arquitectura hexagonal votada por el curso, con interfaz + `Impl` en casos de uso y servicios, y dominio sin frameworks.

- Puerto oficial `8300` (GJ-90), cliente Feign a `ms-item-catalogo` en `8220`, base de datos propia `notificaciones_db` y exchange compartido `styleradar.exchange`.
- Secretos fuera del repositorio (`JwtSecretValidator` bloquea arranque en producción sin `JWT_SECRET`); solo se versiona `.env.example`.
- Cada diagrama acompañado de explicación y justificación; cada tarea cerrada con evidencia verificable.

1.4 ESTADO DE IMPLEMENTACIÓN POR SPRINT

Al cierre del Sprint 2, la totalidad del alcance de los Sprints 1 y 2 se encuentra implementada, probada y verificada en el pipeline CI/QA.

SPRINT	TAREAS GJ	ALCANCE PLANIFICADO	ESTADO AL CIERRE DEL SPRINT 2
Sprint 1	GJ-21 a GJ-29	Repositorio, scaffolding, Docker, CI/CD, Swagger, README, diagramas, documento v1 y análisis.	**Completado (100 %).**
Sprint 2	GJ-30 a GJ-36, GJ-45 a GJ-52, GJ-90 a GJ-92, GJ-99	Contratos, dominio, puertos, JPA, endpoints, JWT, consumidores (`stockAnterior`), Feign (`503`), idempotencia (`V3`), 3 reintentos + DLQ, puerto `8300`, broker Azure TLS `:5671`, Newman y documento v2.	**Implementado y verificado en CI (commit `f88c0cd`).** Pendiente únicamente aprobación final de nombres de eventos en matriz (GJ-30).
Sprint 3	GJ-59 a GJ-63	Notificación generalizada, leídas y contador, nuevos tipos, SSE y evento `AlertaCreada`.	**Implementado en código**; cierre formal en el tablero durante el Sprint 3.
Sprint 3	GJ-64, GJ-65	Preferencias de canal (`preferencias_avisos`), correo (`JavaMailSender`) y aviso de pago confirmado.	**Planificado para Sprint 3.**

02 ARQUITECTURA HEXAGONAL

Las dependencias apuntan hacia el dominio. Los casos de uso son los puertos de entrada; repositorios, publicadores y clientes externos son los puertos de salida.

2.1 DIAGRAMA DE COMPONENTES

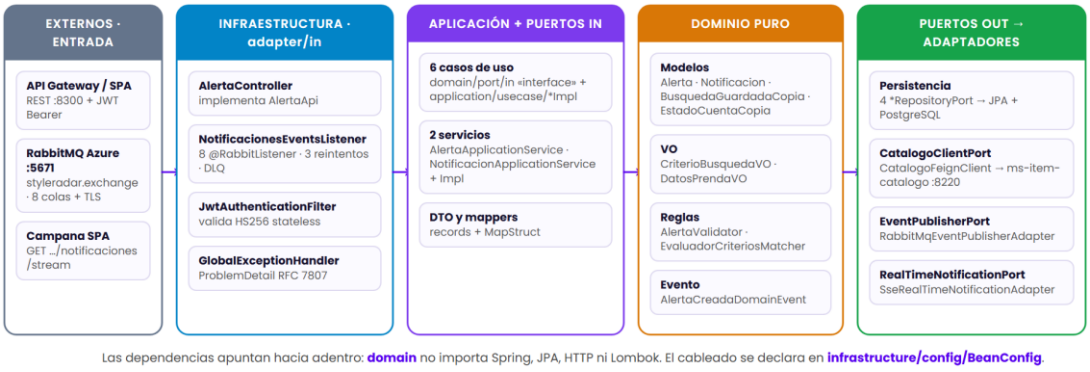


Figura 1. Componentes de GJ-Notification-Service (:8300) al cierre del Sprint 2.

Explicación: los adaptadores de entrada traducen peticiones HTTP (`:8300`) y mensajes AMQPS (`:5671`) a llamadas sobre interfaces de casos de uso. La capa de aplicación coordina el flujo, el dominio puro evalúa las reglas (`AlertaValidator`, `EvaluadorCriteriosMatcher`) y los adaptadores de salida resuelven persistencia en PostgreSQL, consulta a `ms-item-catalogo` (`:8220`), publicación AMQP y emisión SSE.

Justificación: aislar el dominio permite probar las reglas de negocio sin contexto de Spring y evolucionar la infraestructura (como el paso a RabbitMQ con TLS y reintentos en `RabbitRetryConfig`) sin tocar las entidades de dominio.

2.2 CÓMO VIAJA LA CREACIÓN DE UNA ALERTA

```
AlertaController (POST /api/v1/usuarios/me/alertas :8300)
  → CrearAlertaUseCase «interface» → CrearAlertaUseCaseImpl
    → AlertaApplicationService «interface» → AlertaApplicationServiceImpl
        → AlertaValidator                valida reglas de dominio (400 / 422)
        → CatalogoClientPort             → CatalogoClientAdapter → ms-item-catalogo (:8220, 422 si hay stock /
503 si cae)
        → AlertaRepositoryPort           → AlertaRepositoryAdapter → PostgreSQL (409 si ya existe activa)
        → EventPublisherPort             → RabbitMqEventPublisherAdapter → styleradar.exchange
```

2.3 ESTRUCTURA DE PAQUETES

```
edu.dosw.styleradar.notificaciones
├─ domain/          model · event · exception · validator · port/in · port/out
├─ application/     usecase · service/impl · dto · mapper/in · mapper/out
└─ infrastructure/
   ├─ adapter/in/   rest · rest/doc · amqp (NotificacionesEventsListener)
   ├─ adapter/out/  persistence · external · messaging · security · sse
   └─ config/       BeanConfig · SecurityConfig · JwtSecretValidator · RabbitMqConfig · RabbitRetryConfig ·
OpenApiConfig · GlobalExceptionHandler
```

2.4 DEUDA ARQUITECTÓNICA CONOCIDA Y SEGUIMIENTO

HALLAZGO	CONSECUENCIA	ESTADO / ACCIÓN
`application/mapper/out` importa entidades JPA de `infrastructure`.	La capa de aplicación conoce tipos de persistencia en los mappers MapStruct.	Mantenido por simplicidad de generación MapStruct; el dominio puro (`domain/`) sigue 100 % libre de JPA y Spring.
`AlertaController` invoca `SseRealTimeNotificationAdapter` y `SecurityContextUserExtractor`.	El controlador REST resuelve la extracción del principal JWT y el registro del `SseEmitter`.	Acotado a la capa `infrastructure/adapter`; los casos de uso reciben únicamente el `usuarioId` primitivo.
Unificación de nombres de eventos en matriz del curso (**GJ-30**).	Posible ajuste de alias entre `InventarioActualizado` / `StockChangedEvent` y `PrendaPublicada` / `NuevaPrendaCreadaEvent`.	Pendiente de confirmación de los líderes de Inventario y Catálogo; las colas y routing keys son parametrizables en `application.yml`.

03 ENDPOINTS Y SWAGGER (:8300)

Todos los endpoints de negocio se publican en el puerto `8300` bajo `/api/v1/usuarios/me`, requieren JWT Bearer (`HS256`) y operan sobre el usuario autenticado del token.

3.1 CONTRATO REST

MÉTODO	RUTA	DESCRIPCIÓN	CÓDIGOS
POST	`/alertas`	Crea una alerta de prenda agotada (`ITEM_AGOTADO`) o de búsqueda guardada (`BUSQUEDA_GUARDADA`).	201, 400, 401, 409, 422, 503
GET	`/alertas`	Lista todas las alertas registradas por el usuario autenticado.	200, 401
DELETE	`/alertas/{alertaId}`	Desactiva una alerta propia cambiando su estado a `DESACTIVADA`.	200, 401, 403, 404
PATCH	`/alertas/{alertaId}/desactivar`	Ruta alternativa compatible para desactivar una alerta propia.	200, 401, 403, 404
GET	`/notificaciones`	Lista la bandeja del usuario; admite filtro `?soloNoLeidas=true|false`.	200, 401
PATCH	`/notificaciones/{notificacionId}/leer`	Marca una notificación propia como leída y registra `fechaLectura`.	200, 401, 403, 404
PATCH	`/notificaciones/leer-todas`	Marca todas las notificaciones del usuario como leídas y devuelve el contador en `0`.	200, 401
GET	`/notificaciones/no-leidas/contador`	Devuelve el contador de notificaciones no leídas para el badge de la campana.	200, 401
GET	`/notificaciones/stream`	Abre la suscripción SSE (`text/event-stream`) y emite el evento inicial `connected`.	200, 401

Rutas públicas: `/actuator/health`, `/swagger-ui` (redirige con `302` a `/swagger-ui/index.html`) y `/v3/api-docs`.

3.2 MANEJO DE ERRORES (RFC 7807 PROBLEMDETAIL)

`GlobalExceptionHandler` responde con `ProblemDetail` e incluye `timestamp` y `mensaje`. En el Sprint 2 se añadió el manejo explícito de caída de `ms-item-catalogo` con `503 Service Unavailable` (**GJ-49**).

CÓDIGO	EXCEPCIÓN / ORIGEN	CAUSA EN EL SERVICIO
400	`MethodArgumentNotValidException` / `ReglaDeNegocioException`	Petición malformada o combinación inválida de campos en la alerta.
401	`AuthenticationEntryPoint` (`SecurityConfig`)	Token JWT ausente, con firma inválida o expirado.
403	`AccesoDenegadoException`	El usuario autenticado intenta desactivar una alerta o leer una notificación que pertenece a otro `usuarioId`.
404	`RecursoNoEncontradoException`	La alerta o notificación solicitada no existe o la ruta HTTP no está mapeada.
409	`ConflictoException`	Ya existe una alerta `ACTIVA` idéntica para ese usuario y recurso.
422	`ReglaDeNegocioException`	La prenda consultada en `ms-item-catalogo` todavía tiene stock disponible (solo se permiten alertas sobre ítems agotados, **GJ-49**).
503	`ServicioExternoNoDisponibleException`	`ms-item-catalogo` (`:8220`) no responde o devuelve error al verificar el stock de la prenda.

3.3 EJEMPLO DE PETICIÓN EXITOSA (201) Y DE ALERTA AJENA (403)

```
POST /api/v1/usuarios/me/alertas
Authorization: Bearer <JWT usuarioId=101>
{ "tipoAlerta": "ITEM_AGOTADO", "itemId": 1045, "tallaObjetivo": "M", "canal": "IN_APP" }

201 Created
{ "id": 1, "usuarioId": 101, "tipoAlerta": "ITEM_AGOTADO", "itemId": 1045, "tallaObjetivo": "M",
  "busquedaGuardadaId": null, "canal": "IN_APP", "estado": "ACTIVA",
  "fechaCreacion": "2026-11-07T12:00:00", "fechaActualizacion": "2026-11-07T12:00:00" }

DELETE /api/v1/usuarios/me/alertas/1
Authorization: Bearer <JWT usuarioId=999>

403 Forbidden (application/problem+json)
{ "type": "about:blank", "title": "Acceso denegado", "status": 403,
  "detail": "No tiene permisos para modificar esta alerta", "mensaje": "No tiene permisos para modificar esta
alerta" }
```

3.4 EVIDENCIA DE SWAGGER EN EL PUERTO 8300

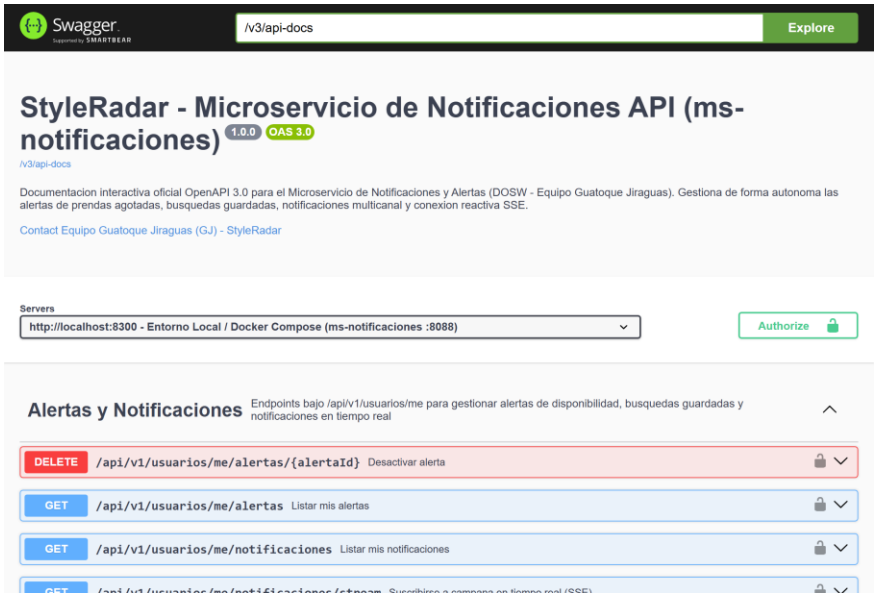


Figura 2. Swagger UI ejecutándose en http://localhost:8300/swagger-ui/index.html (commit f88c0cd).

04 EVENTOS CONSUMIDOS Y PUBLICADOS

La comunicación asíncrona opera sobre **RabbitMQ 3.13.7** desplegado en una VM de Azure (`styleradar-rabbitmq-qa.northcentralus.cloudapp.azure.com`, puerto AMQPS `5671` con TLS, **GJ-91**), usando el exchange durable `styleradar.exchange` (tipo `topic`).

4.1 LOS 8 EVENTOS CONSUMIDOS Y SUS COLAS DURABLES

EVENTO · EMISOR	ROUTING KEY	COLA DURABLE	REGLA DE NEGOCIO EN SPRINT 2
InventarioActualizado · ms-inventario	`styleradar.inventario.actualizado`	`….inventario-actualizado.q`	Verifica reposición real (`stockAnterior <= 0 && stockActual > 0`, **GJ-45**), evalúa alertas y notifica una vez por ciclo.
PrendaPublicada · ms-item-catalogo	`styleradar.catalogo.prenda.publicada`	`….prenda-publicada.q`	Evalúa en memoria (`EvaluadorCriteriosMatcher`, **GJ-46**) contra copias locales de búsquedas guardadas activas.
UsuarioEstadoCambiado · ms-usuarios	`styleradar.usuarios.estado.cambiado`	`….usuario-estado.q`	Sincroniza `estados_cuenta_copia`; bloquea envíos a cuentas `INACTIVA`, `SUSPENDIDA` o `ELIMINADA` (**GJ-48**).
BusquedaGuardadaSincronizada · ms-usuarios	`styleradar.usuarios.busqueda.sincronizada`	`….busqueda-guardada.q`	Sincroniza `busquedas_guardadas_copia`; si la acción es `ELIMINADA`, desactiva sus alertas (**GJ-47**).
MensajeEnviado · ms-chat	`styleradar.chat.mensaje.enviado`	`….mensaje-enviado.q`	Genera notificación `MENSAJE_NUEVO` si la cuenta del destinatario está `ACTIVA`.
DonacionEstadoCambiado · ms-donaciones	`styleradar.donaciones.estado.cambiado`	`….donacion-cambiada.q`	Genera notificación `CAMBIO_DONACION` al donante.
ModeracionPrendaCambiada · ms-item-catalogo	`styleradar.catalogo.moderacion.cambiada`	`….moderacion-cambiada.q`	Notifica al propietario; si el estado es `RETIRADA` o `ELIMINADA`, desactiva alertas del ítem.
PrendaRetirada · ms-item-catalogo	`styleradar.catalogo.prenda.retirada`	`….prenda-retirada.q`	Desactiva todas las alertas activas asociadas al `itemId` retirado.

Nota sobre **GJ-30**: las routing keys y nombres de colas están externalizados en `application.yml` (`application.messaging.*`), listos para reflejar cualquier ajuste acordado con los equipos de Catálogo e Inventario en la matriz oficial del curso.

4.2 EVENTO PUBLICADO

EVENTO · CONSUMIDOR	ROUTING KEY	CUÁNDO SE PUBLICA
AlertaCreada · ms-estadisticas	`styleradar.notificaciones.alerta.creada`	Al persistir exitosamente una alerta activa en `CrearAlertaUseCaseImpl`.

```
{ "eventoId": "evt-alr-1001", "alertaId": 75, "usuarioId": 101, "tipoAlerta": "ITEM_AGOTADO",
  "itemId": 1045, "busquedaGuardadaId": null, "canal": "IN_APP", "fechaCreacion": "2026-11-07T12:30:00" }
```

4.3 BROKER EN AZURE (GJ-91), REINTENTOS E IDEMPOTENCIA (GJ-50)

- **Broker RabbitMQ 3.13.7 en Azure (GJ-91):** desplegado en la VM `vm-styleradar-rabbitmq` (`rg-styleradar-qa-ncus`, `styleradar-rabbitmq-qa.northcentralus.cloudapp.azure.com`) con TLS habilitado en el puerto `5671`,

disco de datos persistente, NSG restringido y usuarios independientes por servicio (`ms_notificaciones`, `ms_catalogo`, `ms_inventario`, `ms_identidad`, `ms_chat`, `ms_donaciones`, `ms_pagos`, sin usuario `guest`).

- **Regla de reposición con `stockAnterior` (GJ-45):** `InventarioActualizadoEventPayload.esReposicion()` verifica que `stockAnterior <= 0 && stockActual > 0` antes de disparar alertas de ítem agotado, ignorando variaciones de stock que no correspondan a una reposición.
- **3 reintentos exponenciales antes de la DLQ (GJ-50):** `RabbitRetryConfig` configura un interceptor stateless con `maxAttempts = 3` (intervalo inicial `1000 ms`, multiplicador `2.0`, máximo `10000 ms`). Los errores de validación de payload lanzan `AmqpRejectAndDontRequeueException` (no reintentable) y van de inmediato a `styleradar.dlx` → `styleradar.notificaciones.dlq` (routing key `styleradar.notificaciones.error`). Las fallas transitorias se reintentan 3 veces antes de enviarse a la DLQ con `RejectAndDontRequeueRecoverer`.
- **Idempotencia transaccional (GJ-50):** la migración `V3__add_deduplication_key.sql` añade la columna `clave_deduplicacion` (`alertaId:itemId:cicloReposicion`) con restricción única `uk_notificacion_deduplicacion`, y `NotificacionRepositoryAdapter` captura `DataIntegrityViolationException` sin abortar el consumidor en entregas concurrentes.

05 MODELO DE DATOS

Servicio con persistencia propia en PostgreSQL 16 (`notificaciones_db`). El esquema se versiona con Flyway en `V1__init_schema.sql`, `V2__add_indexes.sql` y `V3__add_deduplication_key.sql`, y Hibernate solo lo valida (`ddl-auto: validate`).

5.1 ESQUEMA FÍSICO (MIGRACIONES V1, V2 Y V3)

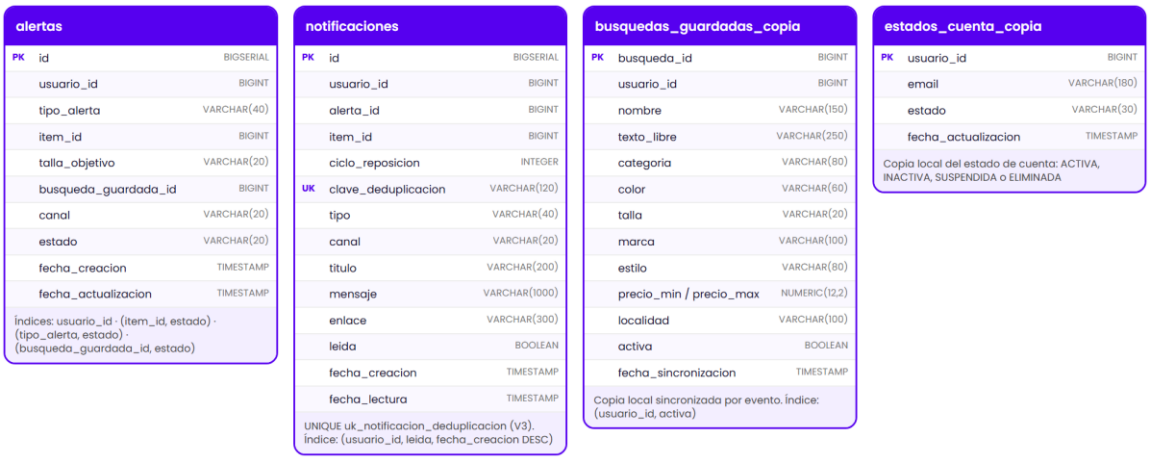


Figura 3. Tablas, restricciones e índices en PostgreSQL 16 (migraciones V1, V2 y V3).

Explicación del diagrama: el modelo físico separa las alertas configuradas por el comprador (`alertas`), el historial e idempotencia de avisos (`notificaciones`) y las dos proyecciones locales sincronizadas por eventos (`busquedas_guardadas_copia` y `estados_cuenta_copia`). Ninguna tabla declara llaves foráneas hacia otros microservicios para respetar *Database-per-Service*.

5.2 DECISIONES DEL MODELO

DECISIÓN	JUSTIFICACIÓN
Sin llaves foráneas externas	`usuario_id`, `item_id` y `busqueda_id` pertenecen a otros microservicios; se almacenan como referencias lógicas `BIGINT`.
Copias locales por evento	`busquedas_guardadas_copia` (**GJ-47**) y `estados_cuenta_copia` (**GJ-48**) permiten evaluar coincidencias y estado de cuenta en milisegundos sin acoplamiento HTTP.
Clave de deduplicación (V3)	`clave_deduplicacion` (`VARCHAR(120)`, `uk_notificacion_deduplicacion`) garantiza exactamente un aviso por alerta, prenda y ciclo de reposición incluso bajo concurrencia.
Criterios tipados e índices	Los criterios de búsqueda se almacenan en columnas tipadas y se indexan los accesos por `(item_id, estado)`, `(tipo_alerta, estado)` y `(usuario_id, leida, fecha_creacion DESC)`.

5.3 VALORES DE DOMINIO

CAMPO	VALORES EN EL CÓDIGO (SPRINT 2)
`TipoAlerta`	`ITEM_AGOTADO`, `BUSQUEDA_GUARDADA`
`TipoNotificacion`	`REPOSICION_STOCK`, `COINCIDENCIA_BUSQUEDA`, `MENSAJE_NUEVO`, `CAMBIO_DONACION`, `MODERACION_PRENDA`, `SISTEMA`
`CanalNotificacion`	`IN_APP`, `EMAIL`, `PUSH` (activo `IN_APP`; `EMAIL` planificado en GJ-64)
`EstadoCuenta`	`ACTIVA`, `INACTIVA`, `SUSPENDIDA`, `ELIMINADA` (solo `ACTIVA` recibe notificaciones, **GJ-48**)

06 DECISIONES TÉCNICAS (ADR)

Cada decisión arquitectónica se evaluó con una matriz multicriterio ponderada y se registra como ADR con las alternativas descartadas.

6.1 CRITERIOS Y ESCALA

ID	CRITERIO	PESO	QUÉ EVALÚA EN ESTE SERVICIO
C1	**Consistencia y resiliencia**	25 %	No duplicar avisos ni perder eventos ante fallos.
C2	**Madurez y estabilidad**	20 %	Soporte, documentación y estabilidad de versiones.
C3	**Velocidad del equipo**	20 %	Curva de aprendizaje y productividad en el curso.
C4	**Operabilidad en Docker y Azure**	15 %	Contenedores, consumo de recursos en VM B2ats_v2 y observabilidad.
C5	**Costo y viabilidad académica**	20 %	Operar dentro de los créditos de Azure for Students y GitHub Actions.

Escala: **5** sobresaliente · **4** bueno · **3** aceptable · **2** deficiente · **1** inaceptable.

6.2 ADR-001 · LENGUAJE Y RUNTIME

CRITERIO (PESO)	JAVA 21 LTS	GO 1.22	NODE.JS 20 + TS	PYTHON 3.12
C1: Consistencia (25 %)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)	●●●○○ (3)
C2: Madurez (20 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)	●●●●○ (4)
C3: Velocidad (20 %)	●●●●○ (4)	●●○○○ (2)	●●●●○ (4)	●●●●○ (4)
C4: Operabilidad (15 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)
C5: Costo (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)
Total ponderado	**4.80 / 5.00 (seleccionado)**	**3.95 / 5.00**	**3.75 / 5.00**	**3.60 / 5.00**

Justificación técnica del equipo: Java 21 aporta tipado estricto, `records` inmutables para payloads de eventos y soporte maduro para Spring AMQP, JPA y JaCoCo.

6.3 ADR-002 · FRAMEWORK

CRITERIO (PESO)	SPRING BOOT 3.2.5	QUARKUS	NESTJS	FASTAPI
C1: Consistencia (25 %)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)	●●●○○ (3)
C2: Madurez (20 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)	●●●●○ (4)
C3: Velocidad (20 %)	●●●●● (5)	●●●○○ (3)	●●●○○ (3)	●●●●○ (4)
C4: Operabilidad (15 %)	●●●●○ (4)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)
C5: Costo (20 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)	●●●●○ (4)
Total ponderado	**4.85 / 5.00 (seleccionado)**	**3.95 / 5.00**	**3.55 / 5.00**	**3.60 / 5.00**

Justificación técnica del equipo: Spring Boot 3.2.5 integra Web, Security, Data JPA, AMQP (con `RetryOperationsInterceptor`), Actuator y OpenFeign manteniendo el dominio desacoplado mediante `BeanConfig`.

6.4 ADR-003 · BASE DE DATOS

CRITERIO (PESO)	POSTGRESQL 16	MYSQL 8	MONGODB 7	DYNAMODB
C1: Consistencia (25 %)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)	●●●○○ (3)
C2: Madurez (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)
C3: Velocidad (20 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)	●●○○○ (2)
C4: Operabilidad (15 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)	●●○○○ (2)
C5: Costo (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●○○○ (2)
Total ponderado	**5.00 / 5.00 (seleccionado)**	**4.40 / 5.00**	**3.75 / 5.00**	**2.65 / 5.00**

Justificación técnica del equipo: PostgreSQL 16 garantiza transacciones ACID y la restricción única `uk_notificacion_deduplicacion` necesaria para la idempotencia por ciclo.

6.5 ADR-004 · BROKER DE MENSAJERÍA EN VM DE AZURE (GJ-91)

CRITERIO (PESO)	RABBITMQ 3.13 EN VM AZURE	APACHE KAFKA	AZURE SERVICE BUS	REDIS PUB/SUB
C1: Consistencia (25 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)	●○○○○ (1)
C2: Madurez (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)
C3: Velocidad (20 %)	●●●●○ (4)	●●○○○ (2)	●●●○○ (3)	●●●●○ (4)
C4: Operabilidad (15 %)	●●●●● (5)	●●○○○ (2)	●●●○○ (3)	●●●●○ (4)
C5: Costo (20 %)	●●●●● (5)	●●●○○ (3)	●●○○○ (2)	●●●●○ (4)
Total ponderado	**4.80 / 5.00 (seleccionado)**	**3.30 / 5.00**	**3.25 / 5.00**	**3.25 / 5.00**

Justificación técnica del equipo: RabbitMQ 3.13.7 en VM de Azure ofrece enrutamiento `topic` (`styleradar.exchange`), DLX/DLQ nativos (`styleradar.dlx` / `styleradar.notificaciones.dlq`), TLS en el puerto `5671` y paridad exacta con el contenedor de `docker-compose.yml`, sin consumir el presupuesto estudiantil como Kafka ni perder mensajes como Redis Pub/Sub.

6.6 ADR-005 · TIEMPO REAL PARA LA CAMPANA

CRITERIO (PESO)	SSE	WEBSOCKET + STOMP	LONG POLLING	SHORT POLLING
C1: Consistencia (25 %)	●●●●○ (4)	●●●●○ (4)	●●●○○ (3)	●●○○○ (2)
C2: Madurez (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●●● (5)
C3: Velocidad (20 %)	●●●●● (5)	●●●○○ (3)	●●●○○ (3)	●●●●● (5)
C4: Operabilidad (15 %)	●●●●● (5)	●●●○○ (3)	●●●○○ (3)	●●●●○ (4)
C5: Costo (20 %)	●●●●● (5)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)
Total ponderado	**4.75 / 5.00 (seleccionado)**	**4.05 / 5.00**	**3.40 / 5.00**	**3.70 / 5.00**

Justificación técnica del equipo: La campana solo requiere envío unidireccional servidor → cliente; **SSE** (`SseEmitter`) opera sobre HTTP estándar y las acciones de lectura usan `PATCH` REST.

6.7 ADR-006 · CORREO ELECTRÓNICO (PLANIFICADO SPRINT 3, GJ-64)

CRITERIO (PESO)	JAVAMAILSENDER	SENDGRID SDK	AWS SES SDK
C1: Consistencia (25 %)	●●●●○ (4)	●●●●○ (4)	●●●●○ (4)
C2: Madurez (20 %)	●●●●● (5)	●●●●○ (4)	●●●●○ (4)
C3: Velocidad (20 %)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)
C4: Operabilidad (15 %)	●●●●● (5)	●●●●○ (4)	●●●○○ (3)
C5: Costo (20 %)	●●●●● (5)	●●●○○ (3)	●●●○○ (3)
Total ponderado	**4.75 / 5.00 (seleccionado)**	**3.80 / 5.00**	**3.45 / 5.00**

Justificación técnica del equipo: `JavaMailSender` abstrae el servidor SMTP detrás de un puerto de salida; su implementación está programada para la historia **GJ-64** en el Sprint 3.

07 RIESGOS Y MITIGACIÓN TÉCNICA

Evolución de los riesgos técnicos identificados en la v1 y estado de mitigación al cierre del Sprint 2.

7.1 EVALUACIÓN Y ESTADO DE MITIGACIÓN

RIESGO IDENTIFICADO EN V1	PROB.	IMPACTO	ESTADO AL CIERRE DEL SPRINT 2
Secreto JWT por defecto fuera de desarrollo	Baja	Crítico	**Mitigado (GJ-35):** `application.yml` no tiene secreto por defecto en prod y `JwtSecretValidator` detiene el arranque si `JWT_SECRET` está vacío o usa el valor de desarrollo.
Catálogo no disponible al crear una alerta	Media	Medio	**Mitigado (GJ-49):** `CatalogoClientAdapter` lanza `ServicioExternoNoDisponibleException` y el API responde `503 Service Unavailable` (no crea alertas sobre stock desconocido).
Fallo transitorio enviado a DLQ sin reintentos	Baja	Medio	**Mitigado (GJ-50):** `RabbitRetryConfig` aplica 3 intentos con backoff exponencial (`1 s`, `2 s`, `4 s`) antes de enrutar a `styleradar.notificaciones.dlq`.
Colisión concurrente al guardar notificación duplicada	Baja	Medio	**Mitigado (GJ-50):** migración `V3` (`clave_deduplicacion`) y captura limpia de `DataIntegrityViolationException` probada con hilos concurrentes (`NotificacionesConcurrenciaTest`).
Pruebas solo sobre H2 sin verificar PostgreSQL ni RabbitMQ reales	Baja	Medio	**Mitigado (GJ-52 / GJ-92):** el job `docker-verify-qa` en `ci-qa.yml` levanta PostgreSQL 16 + RabbitMQ 3.13.7 + `ms-notificaciones` y ejecuta las 8 pruebas de Newman y la prueba de DLQ.
Despliegue en Azure QA pendiente de secretos en GitHub	Media	Alto	**En cierre (GJ-91 / GJ-92):** broker RabbitMQ en Azure VM ya activo con TLS (`:5671`); workflows `ci-qa.yml` y `cd-prod.yml` listos para publicar en Docker Hub y desplegar en App Service.

7.2 CONTROLES DE CALIDAD VIGENTES EN EL SPRINT 2

PIPELINE CI + NEWMAN

`ci-qa.yml` ejecuta `mvn clean verify` (JaCoCo >= 85 %), levanta `docker-compose.yml` y corre la colección Postman con Newman (8/8 requests, 15/15 aserciones en verde).

BROKER AZURE TLS :5671

RabbitMQ 3.13.7 en `vm-styleradar-rabbitmq` con TLS (`tlsv1.2`), 8 colas durables, `styleradar.dlx`, `styleradar.notificaciones.dlq` y usuarios por servicio.

RESILIENCIA Y RFC 7807

3 reintentos exponenciales AMQP, deduplicación por ciclo (`V3`) y respuestas de error estandarizadas con `ProblemDetail` (400, 401, 403, 404, 409, 422, 503).

08 CONCLUSIONES Y EQUIPO DE TRABAJO

8.1 STACK TECNOLÓGICO CON VERSIONES FIJAS

CAPA	TECNOLOGÍA Y VERSIÓN	ROL EN EL MICROSERVICIO (SPRINT 2)
Lenguaje	**Java 21 LTS** (Eclipse Temurin)	`records`, tipado estricto y dominio puro.
Framework	**Spring Boot 3.2.5** · Spring Security 6.2.4	API REST en `:8300`, seguridad JWT, Actuator e inyección de dependencias.
Persistencia	**PostgreSQL 16** · Hibernate 6.4.4 · Flyway 9.22.3	Base de datos `notificaciones_db`, migraciones `V1`–`V3` y `uk_notificacion_deduplicacion`.
Mensajería	**RabbitMQ 3.13.7** (Azure VM TLS `:5671`) · Spring AMQP 3.1.4	Exchange `styleradar.exchange`, 8 colas durables, 3 reintentos y DLQ `styleradar.notificaciones.dlq`.
Cliente HTTP	**OpenFeign** (Spring Cloud 2023.0.1)	Consulta síncrona de stock en `ms-item-catalogo` (`:8220`) con manejo de `503`.
Tiempo real	**SSE** con `SseEmitter`	Stream de notificaciones en `/api/v1/usuarios/me/notificaciones/stream`.
Seguridad	**JJWT 0.11.5** (HS256) + `JwtSecretValidator`	Autenticación stateless y validación obligatoria de secreto en producción.
Mapeo	**MapStruct 1.5.5** · Lombok 1.18.48	Conversión entre DTO, modelos de dominio y entidades JPA.
Documentación	**springdoc-openapi 2.5.0**	Swagger UI en `/swagger-ui` y contrato OpenAPI 3.0 en `/v3/api-docs`.
Calidad y API	**JUnit 5** · Mockito · WireMock · **JaCoCo 0.8.11** · **Newman 6.2.1**	Cobertura >= 85 % en `mvn verify` y 15 aserciones Postman en CI.
Contenedores	**Docker** multi-stage · Docker Compose	Imagen `styleradar/ms-notificaciones` no-root en puerto `8300`.

8.2 VERIFICACIÓN REALIZADA AL CIERRE DEL SPRINT 2

COMPROBACIÓN	RESULTADO OBTENIDO	FECHA
`mvn -B clean verify` (commit `f88c0cd`)	Suite unitaria y de integración sin fallos · Gate JaCoCo >= 85 % superado	7 nov 2026
GitHub Actions `ci-qa.yml` (run `42345059191`)	`build-and-verify` y `docker-verify-qa` en verde (Docker Compose + DLQ + Newman 8/8 requests, 15/15 aserciones)	7 nov 2026
Broker RabbitMQ 3.13.7 en Azure VM (`GJ-91`)	Conexión `ms_notificaciones` activa por TLS (`tlsv1.2`, puerto `5671`), 8 colas durables y caída a `styleradar.notificaciones.dlq` verificadas	7 nov 2026
Swagger UI y Healthcheck en puerto `8300` (`GJ-90`)	`GET /actuator/health` responde `200 UP` y `/swagger-ui/index.html` responde `200 OK`	7 nov 2026

8.3 PENDIENTES PARA EL SPRINT 3

- **GJ-30:** confirmar con los líderes de Catálogo e Inventario si la matriz final mantiene los nombres `InventarioActualizado` / `PrendaPublicada` o adopta `StockChangedEvent` / `NuevaPrendaCreadaEvent`.
- **GJ-92:** completar la carga de secretos de Docker Hub y Azure App Service QA en GitHub Secrets para activar el paso condicional de publicación y despliegue en QA al fusionar el PR #3 en `develop`.
- **GJ-64 y GJ-65 (Sprint 3):** implementar la tabla `preferencias_avisos`, el adaptador de correo con `JavaMailSender` y el consumidor de pago confirmado.

8.4 RESPONSABLES DEL MICROSERVICIO

NOMBRE	ROL
Paula Diaz	Líder de Squad
Jose Martinez	Líder de Backend
Oscar Lasso	Desarrollo e infraestructura del microservicio de notificaciones